CHAPTER 1

DESIGNING A CAR RENTAL SYSTEM

1 Getting Ready: The Car Rental System 2
2 Requirements for the Car Rental System 4
3 Use Case Diagram for the Car Rental System 6
4 Class Diagram for the Car Rental System 11
5 Sequence Diagram for the Car Rental System 27
6 Activity Diagram for the Car Rental System 30
7 Code for the Car Rental System 31
8 Summary: Designing a Car Rental System 46
9 Key Insights and Tips: Designing a Car Rental System 50

This chapter teaches you the design of a car rental system, detailing requirements, UML diagrams, and implementation code for a functional system.

1 Getting Ready: The Car Rental System

A **car rental system** enables customers to rent vehicles for short periods, ranging from a few hours to several weeks. Such systems are commonly operated by companies with multiple offices across cities, often near airports or commercial centers. The platform allows customers to reserve, pick up, and return vehicles at different locations, depending on availability and convenience.

Each branch manages its fleet, consisting of various vehicle types and models, and tracks reservations, customer information, payments, and the logistical aspects of vehicle movement between locations. Customers can search for available vehicles by location, type, or features and reserve or cancel rentals subject to company policies. The system supports multiple payment methods and handles cases such as late returns, reservation modifications, and optional services (e.g., driver assignment and roadside assistance).

This scenario presents several design challenges, including distributed inventory management, concurrent reservations, flexible pickup and drop-off options, dynamic pricing, and ensuring a seamless and reliable customer experience across all locations.

In this LLD interview case study, you'll focus on:

- Managing vehicle inventory and supporting multiple vehicle types and features.
- Handling reservations, cancellations, and modifications across branches.
- Facilitating vehicle search, pickup, and drop-off at different locations.
- Tracking customer information, rental history, and payments.
- Supporting optional services and flexible payment methods.
- Ensuring system reliability, data consistency, and scalability.

This system model can be adapted for any rental business managing distributed assets and flexible reservations.

Figure 1.1: Application of the car rental system

1.1 Expectations from the interviewee

The car rental system consists of multiple components with functionality and constraints. This section provides an overview of some of the main expectations that the interviewer will want to hear you discuss in more detail during the interview.

Vehicle types

An interviewer would expect you to discuss the different vehicle types and ask the following questions:

- What types of vehicles will that system support?
- How can we identify the specific vehicle?

Search interface

Members will use the application and add the location and the reservation date. They will receive several options to select the vehicle. Therefore, an interviewer would expect you to ask the questions listed below:

- Is it possible to search for a vehicle using its name or type?
- Can we search for a vehicle by its model number?

Services

An interviewer would also expect you to discuss the services of the car rental system and may ask the following questions:

- Does a car rental system assign a driver to its customer?
- Does a car rental system provide roadside assistance to its customers?

Reservation cancellation

There will be many duplicate instances in our system. The interviewer expects you to ask the questions listed below:

- Can the member be able to cancel a reservation?
- Which member is allowed to request a vehicle reservation cancellation and when?

Payment flexibility

One of the car rental system's most significant attributes is its customer payment structure. The payment depends on the vehicle type and the time stamp. Therefore, an interviewer would expect you to ask the questions listed below:

- How can customers pay at different branch locations and by different methods (cash, credit, or cheque)?
- If there are multiple branches of the car rental system, how will the system keep track of customers who have already paid at a particular branch?

1.2 Design approach

We'll design this car rental system using the bottom-up design approach. For this purpose, we will follow the steps below:

1. **Identify core components:** Start with basic entities like `Vehicle`, `Customer`, `Reservation`, `Payment`, etc.

2. **Compose subsystems:** Build larger modules for inventory management, reservation management, and payment processing.

3. **Integrate subsystems:** Assemble the complete application, ensuring the design is modular and adheres to SOLID principles.

4. **Document assumptions:** Clearly state and justify your assumptions where requirements are unclear.

This design approach will address concurrency and edge cases and follow SOLID principles to ensure scalability and maintainability. Later , diagrams and code will illustrate major workflows and class structures.

1.3 Design pattern

During an interview, discussing the design patterns the car rental system falls under is always a good practice. Stating the design patterns gives the interviewer a positive impression and shows that the interviewee is well-versed in the advanced concepts of object-oriented design. Try to answer the following question. If you are not familiar with design patterns, don't worry! You can learn about them by asking, "Define design patterns."

Question:

Which design pattern(s) should be used to design a car rental system? Please elaborate on your choice(s).

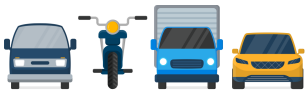
Let's explore the requirements of the car rental system in the next lesson.

2 Requirements for the Car Rental System

In this lesson, we outline the functional and operational requirements for the car rental system. Identifying and understanding requirements is essential to defining the system's scope and ensuring a robust, user-friendly design.
We'll use the notational convention to identify each requirement with a unique label "Rn," where "R" is short for requirement and "n" is a natural number.

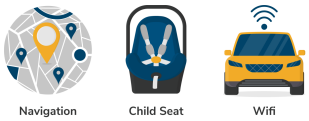
2.1 Requirement collection

The set of requirements for the car rental system is listed below:

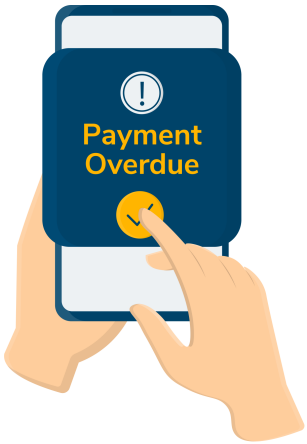


- **R1:** The system supports two main user roles: customers and receptionists.
- **R2:** The system manages multiple types of vehicles, including cars, trucks, vans, and motorcycles.
- **R3:** Each vehicle type may have multiple subtypes, such as:
  - **Cars**: Economy, Luxury, Standard, Compact, Intermediate, Full size, Premium
  - **Vans**: Passenger, Cargo
  - **Motorcycles**: Standard, Cruiser, Touring, Sports, Off-road, Dual purpose
  - **Trucks**: Light-duty, Medium-duty, Heavy-duty
- **R4:** The system must record every reservation, including the customer details and the date/time a vehicle is issued.
- **R5:** The system can track and report the number of vehicles each customer has rented, including rental history and active reservations.
- **R6:** Customers can cancel their reservations at any time before the pickup date, subject to company policies.
- **R7:** The system maintains a vehicle log to track all significant events related to each vehicle (e.g., maintenance, repairs, accidents, assignments, or status changes).

- **R8:** Customers can add equipment to their reservations, such as a ski rack, child seat, or navigation system.



- **R9:** Customers can add extra services to their reservations, including a driver, Wi-Fi, or roadside assistance.
- **R10:** If a vehicle is not returned by the due date, the system must notify the customer and automatically generate a fine according to company policy.



- **R11:** Users can search for vehicles by type, model, features, or availability at specific locations and dates.
- **R12:** The system must support the management of multiple branches in different cities and locations
- **R13:** Each branch must maintain a record of parking stalls for vehicles at that location, including current status (occupied, available, reserved).
- **R14:** The system should securely process payments, refunds, and fines using multiple payment methods (cash, card, online).
- **R15:** The customer shall be able to make payment for the car rental in both scenarios: when making the reservation and when returning the vehicle. The system should validate both.

We've identified our requirements for the problem. In the next lesson, we'll define different use cases of our car rental system.

3 Use Case Diagram for the Car Rental System

Let's build the use case diagram for the car rental system and understand the relationship between its main actors and system functions. First , we'll define the different elements of our system, followed by the complete use case diagram and a detailed breakdown of actors and use cases.

3.1 System

Our system is the car rental system. It manages automated vehicle reservations, returns, payments, and services for customers across multiple branches.

3.2 Actors

Now, we'll define the main actors of our car rental system.

Primary actor

- **Customer:** The primary user, able to register, search vehicles, make and manage reservations, pick up and return vehicles, and make payments.

Secondary actor

- **Receptionist:** The system operator who can perform all customer operations on behalf of a customer (e.g., register a customer, search, make/update/cancel reservations, and process payments), manage vehicle inventory, and update vehicle logs.

3.3 Use cases

This section defines the use cases for the car rental system. We have listed the use cases according to their respective interactions with a particular actor.

> **Note:** Some use cases will occur multiple times because they are shared among different actors in the system.

Customer

- **Register account:** To create a new customer profile with personal information in the system.
- **Login/Logout:** To securely sign in or out of the system.
- **Search vehicle inventory:** To search and view available vehicles by type, model, location, or availability.
- **Make reservation:** To reserve a vehicle for specific dates and locations, optionally selecting equipment and services.
- **Update reservation:** To request to add additional services such as ski rack, child seat, etc.
- **Cancel reservation:** To cancel an existing reservation before the scheduled pickup time.
- **Pick up vehicle:** To complete check-in, collect a reserved vehicle from the designated branch or location.

- **Return vehicle:** To complete checkout, return a rented vehicle to an authorized branch.
- **Pay bill:** To make payment for rental charges, including any extra services, fines, or late fees.

Receptionist

The following are a few use cases on behalf of customers:

- **Register account:** To create a new account for customers.
- **Login/Logout:** To securely sign in or out of the system.
- **Search vehicle inventory:** To search and view available vehicles by type, model, location, or availability, on behalf of a customer.
- **Make reservation:** To reserve a vehicle for specific dates and locations, optionally selecting equipment and services on behalf of a customer.
- **Update reservation:** To add additional services such as ski rack, child seat, etc., on behalf of a customer.
- **Cancel reservation:** To cancel an existing reservation before the scheduled pickup time on behalf of a customer.
- **Add vehicle:** To register a new vehicle in the branch inventory, specifying type and details.
- **Remove vehicle:** To remove a vehicle from the inventory (e.g., for maintenance or decommission).
- **Modify vehicle info:** To update vehicle information or status (e.g., availability, service state).
- **Update reservation:** To change details of an existing reservation, such as dates, vehicle type, or selected services.
- **Update log:** To record or update significant vehicle events, such as maintenance, repairs, or status changes.
- **Collect bill/payment:** To accept and process customer payment upon vehicle return.

Car rental system

- **Send reservation notification:** To automatically send reservation confirmations to customers.
- **Send reservation cancellation notification:** To notify customers when a reservation is canceled.

- **Send overdue notification:** To notify customers about overdue vehicle returns and any associated fines.

3.4 Relationships

This section describes and justifies the system's relationships between actors and use cases. Understanding why each relationship is used will help clarify system behaviors, support extensibility, and avoid redundancy in the design.

Generalization

We'll use the generalization relationship to add, remove, or modify a vehicle. However, each vehicle type (Car, Truck, Van, Motorcycle) may have unique attributes or business rules. By using generalization:

- The system models "Add vehicle" as the common behavior.
- Each specialized use case ("Add car", "Add truck", etc.) inherits from the general use case and can include additional, type-specific steps.
- This approach improves maintainability and scalability; only a new specialized use case must be added if a new vehicle type is introduced.
- Examples:
  - "Add vehicle" generalizes to "Add car", "Add truck", "Add van", and "Add motorcycle".
  - "Remove vehicle" generalizes to "Remove car", "Remove truck", "Remove van", and "Remove motorcycle".
  - "Modify vehicle" generalizes to "Modify car", "Modify truck", "Modify van", and "Modify motorcycle".

Associations

The table below shows the association relationship between actors and their use cases.

Customer	**Receptionist**	**Car Rental System**
Register account	Register account	Send a reservation canceled notification
Login/Logout	Login/Logout	Send a reservation notification
Search vehicle inventory	Search vehicle inventory	Send overdue notification
Make reservation	Make reservation	
Update reservation	Update reservation	
Cancel reservation	Cancel reservation	

Pick up vehicle	Add vehicle
Pay bill	Modify vehicle info
Return vehicle	Remove vehicle
	Update log
	Collect the bill/payment

Include

The "include" relationship is used when a use case always requires the execution of another use case as part of its flow. This avoids duplicating logic and ensures consistency.
For example, whenever a customer returns a vehicle or cancels a reservation, it is always necessary to process payment for the rental, including any outstanding fees (such as late fines, damages, or extra services). By modeling "Pay bill" as an included use case:

- The return and cancellation processes consistently enforce billing.
- Any changes to payment handling are managed in a single place (the included use case).

Examples:

- "Return vehicle" includes "Pay bill" (the billing/payment process always occurs when returning a vehicle).
- "Make reservation" includes "Send reservation notification" (a confirmation is always sent after a successful reservation).
- "Cancel reservation" includes "Send reservation cancellation notification" (a notification is always sent when a reservation is canceled).
- "Return vehicle" includes "Send overdue notification" (an overdue notification is sent if the vehicle is returned late).

Extend

An "extend" relationship allows optional or conditional behavior to be attached to a base use case. The extension use case is only executed if certain criteria are met.
For instance, when making or updating a reservation, customers may add extras—such as rental insurance, an additional driver, Wi-Fi, or a child seat—but these are not always required. Using "extend":

- The base reservation process remains focused and uncluttered.
- Optional features can be added or changed easily, without modifying the main flow.

Examples:
"Update reservation" extends to the following:

- "Add rental insurance"
- "Add additional driver"
- "Add service" (e.g., Wi-Fi, driver, roadside assistance)
- "Add equipment" (e.g., child seat, ski rack, navigation system)

These extensions are invoked only if the customer requests them during the reservation or update process.

3.5 Use case diagram

Here's the use case diagram for the car rental system:

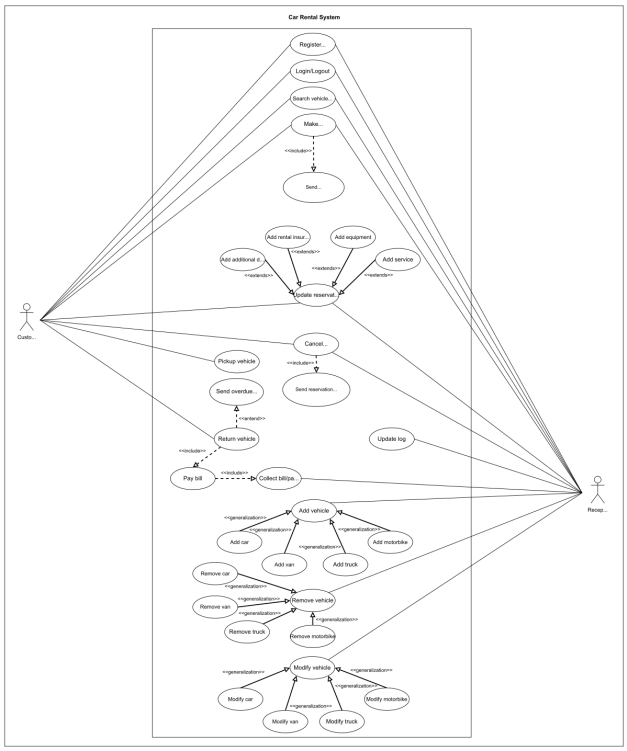


Figure 1.2: The use case diagram of the car rental system

In the next lesson, we'll discuss the class diagram with a detailed explanation of all classes and their relationship with each other.

4 Class Diagram for the Car Rental System

Based on the given requirements, we'll create the class diagram for the car rental system. In the class diagram, we will first identify the system's classes (concrete, abstract, or associated)

and interfaces. Then , we will determine the relationship between them according to the requirements in the previous lesson.

4.1 Components of a car rental system

As mentioned, we'll design the car rental system using a bottom-up approach.

Address and person

The `Address` class is a reusable value object representing a physical address within the car rental system. This class standardizes storing and referencing addresses for customers, staff, and branches. It contains attributes such as street address, city, state/province, postal/zip code, and country.
The `Person` class is an abstract base class that encapsulates common personal information for all individuals in the system, including customers, receptionists, and workers. This class centralizes attributes and methods shared by everyone in the system, promoting code reuse and maintainability. The class representation of `Address` and `Person` is given below:
No change in the class representation of these classes.

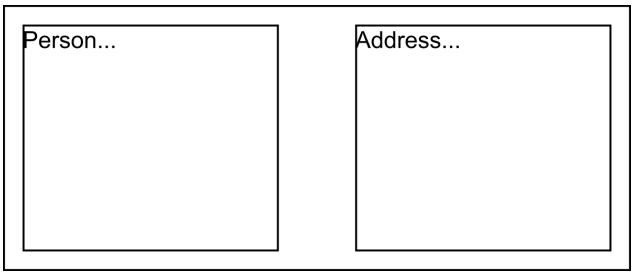


Figure 1.3: The class diagram of the Address and Person classes

R1: The system supports two main user roles: customers and receptionists.
R4: The system must record every reservation, including the customer details and the date/-time a vehicle is issued.
R12: The system must support the management of multiple branches in different cities and locations

Account

The `Account` class represents a person's login and system access credentials. It is an abstract class because only specific user roles (customer, receptionist, worker) log into the system. The `Account` class manages authentication, authorization, and account state. This class has members like account ID, password, account status, etc.
The `Customer` class represents customers who can register, log in, make vehicle reservations, manage their bookings, make payments, and receive notifications. The `Receptionist` is responsible for managing vehicles, reservations, and branch operations. Receptionists inherit all `Customer` privileges and can administer inventory and operations for their branch.
The class representation of `Account` and its subclasses is given below:

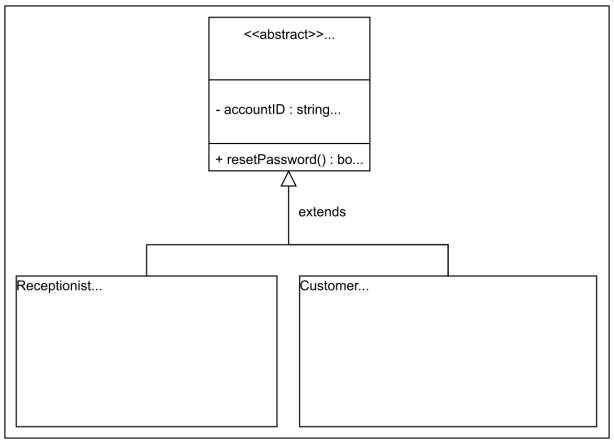


Figure 1.4: The class diagram of the Account class

R1: The system supports two main user roles: customers and receptionists.
R4: The system must record every reservation, including the customer details and the date/-time a vehicle is issued.
R6: Customers can cancel their reservations at any time before the pickup date, subject to company policies.

Driver

We will have a `Driver` class as we are designing the car rental problem. A customer can request an additional driver at the time of reservation. The class diagram is shown below:

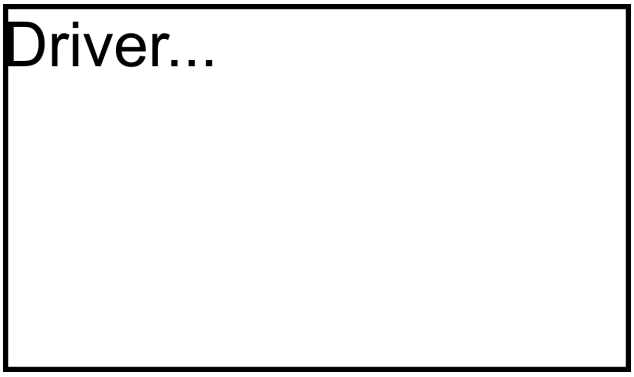


Figure 1.5: The class diagram of the Driver class

Vehicle

Our car rental system should have a vehicle object according to the requirements. The vehicle can be of four types: a car, a truck, a van, or a motorcycle. For this purpose, we'll create `Vehicle` as an abstract class and `Car`, `Truck`, `Van`, and `Motorcycle` as its subclasses, as shown in the figure below:

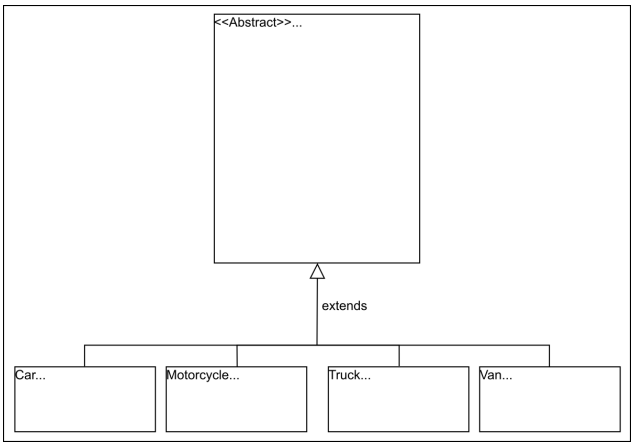


Figure 1.6: The class diagram of Vehicle and its derived classes

R2: The system should handle multiple types of vehicles. Initially, it should cater to cars, trucks, vans, and motorcycles.

Equipment

`Equipment` is an abstract class that stores information about different types of equipment that can be added to the reservation. For simplicity, we'll assume three types of equipment, i.e., navigation, child seat, and ski rack. The class diagram for `Equipment` and its subclasses is as follows:

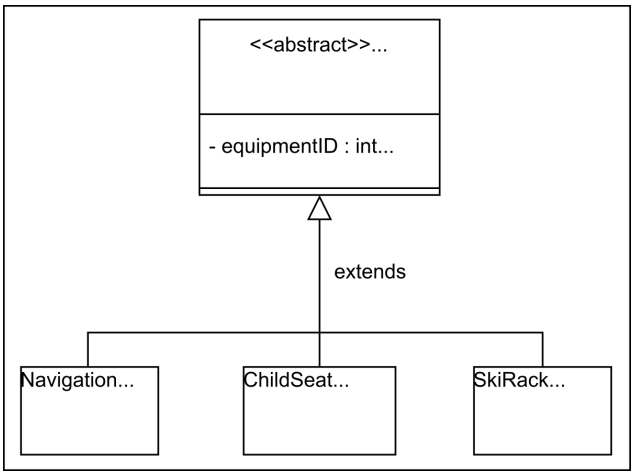


Figure 1.7: The class diagram of Equipment and its derived classes

R8: The system should allow the users to add equipment to the reservations, like a ski rack, child seat, and navigation.

Service

`Service` is an abstract class that represents the services provided to the customers along with the vehicle. While reserving a vehicle, the customers can add a service to their reservation.

Every service has its fixed cost. We have three types of services, i.e., driver, roadside assistance, and Wi-Fi. The UML diagram of `Service`, along with its subclasses `DriverService`, `RoadsideAssistance`, and `Wi-Fi`, is given below:

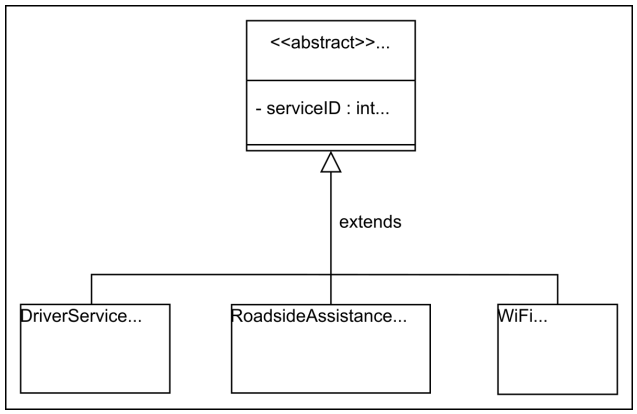


Figure 1.8: The class diagram of Service and its derived classes

R9: The system should allow users to add services to the reservations, like a driver, Wi-Fi, and roadside assistance.

Notification

`Notification` is an abstract class responsible for sending notifications to customers. Every notification has an ID, creation date, and content. The notification can either be an SMS notification or an email notification. The `SMSNotification` class requires the customer's phone number to send a notification, while `EmailNotification` is sent to the customer's email address. The relationship diagram of these classes is shown below:

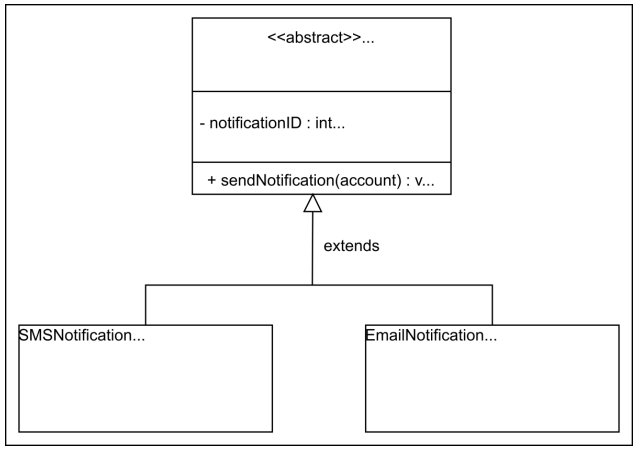


Figure 1.9: The class diagram of Notification and its derived classes

R10: The system should send a notification to the customer and generate a fine if the vehicle is not returned within the due date.

Parking stall

Each car rental location has parking stalls where the vehicles are parked. Each parking stall is identified by its ID, and a location identifier specifies its location. The representation of the `ParkingStall` class is shown below:

Figure 1.10: The class diagram of the ParkingStall class

R13: Every branch of the car rental system should have parking stalls to park the vehicles.

Vehicle log

`VehicleLog` is a class that is used to keep track of all the events related to a vehicle. Every vehicle log has its ID, log type, description, and creation date, as shown here:

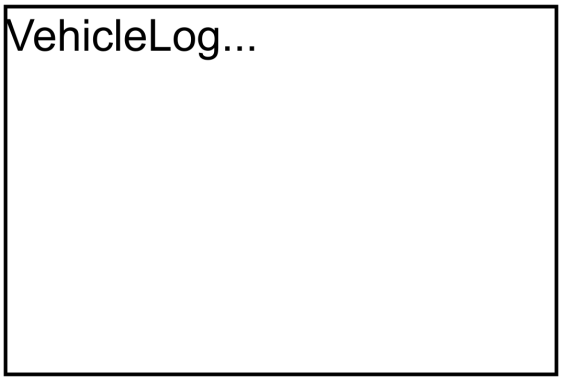


Figure 1.11: The class diagram of the VehicleLog class

R7: To keep track of all events related to the vehicle, the system should maintain a vehicle log.

Vehicle reservation

Vehicle reservation is one of the most important requirements of the car rental system. To fulfill this functionality, we have a `VehicleReservation` class. This class is responsible for managing the vehicle reservation status of vehicles. The customer can also add any equipment or service at the time of reservation. The UML representation of the class is shown below:

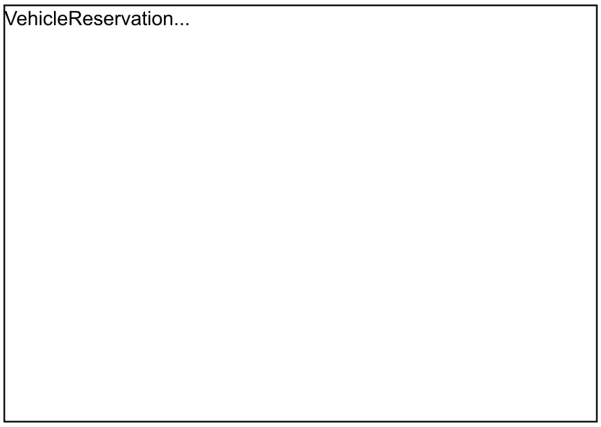


Figure 1.12: The class diagram of the VehicleReservation class

R4: The system should be able to keep a record of who reserved a particular vehicle and on which date the vehicle was issued.

Payment

The `Payment` class will be an abstract class with two child classes: `CreditCard` and `Cash`. These represent the two payment methods in the car rental system. The representation of these classes is given below:

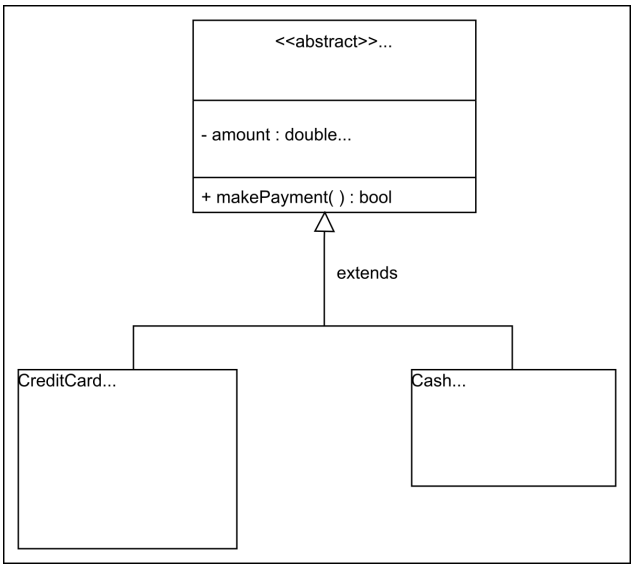


Figure 1.13: The class diagram of Payment and its child classes

Fine

The system needs the `Fine` class to calculate the fine on the vehicle reservation in case the customer returns the vehicle after the due date, the fuel in the vehicle is less than the limit

value, or there is any damage to the vehicle. The representation of this class is given below:

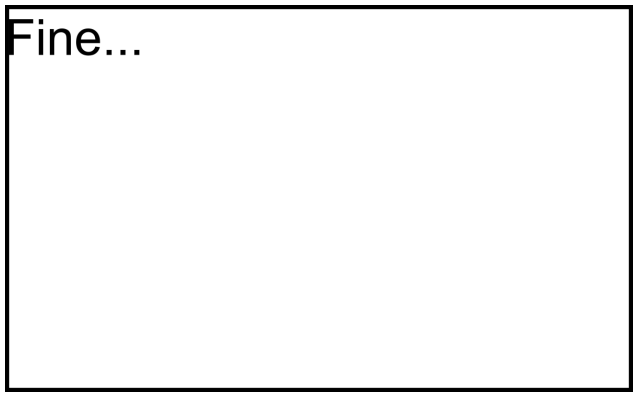


Figure 1.14: The class diagram of the Fine class

R10: If the vehicle is not returned by the due date, the system should send a notification to the customer and generate a fine.

Search interface and vehicle inventory class

`Search` is one of the most important functionalities of the system. It is the interface that allows the user to search for any vehicle and return the list of vehicles upon searching by any of the following methods:

- Search for a car by its type
- Search for a car by its model

The `VehicleCatalog` is a class where the search function is implemented. In each catalog, the vehicles are sorted according to one search technique, i.e., vehicle type or model. The following UML diagram shows this relationship:

R11: The system should allow users to search vehicles by type, model, or seat capacity.

Car rental system and branch

`CarRentalSystem` is the main class of the car rental system and is the central part of the design. There can be multiple branches and locations of the car rental system. The `CarRentalBranch` class will represent each of these branches. The class representation is as follows:

R12: A system should be able to manage the multiple branches of the car rental system.

Enumerations

The list of enumerations required in the car rental system is provided below:

- `VehicleStatus:` The vehicle status describes the status of the particular vehicle for the user, whether it is available, reserved, lost, or serviced.

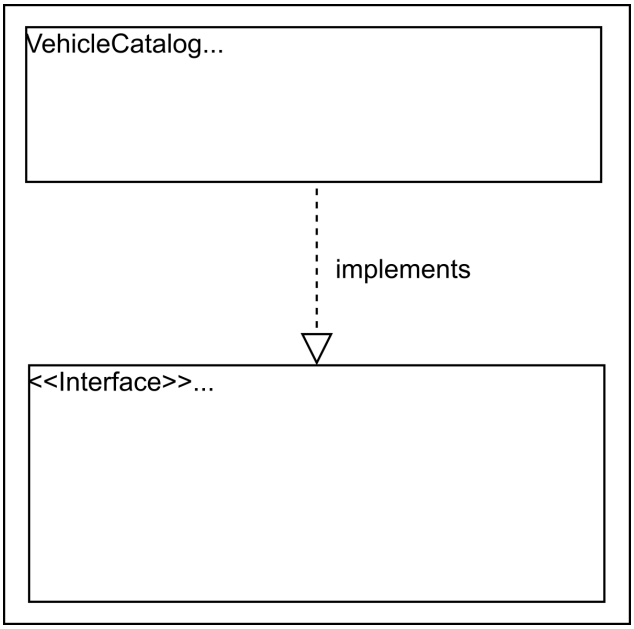


Figure 1.15: The class diagram of the Search interface and the VehicleCatalog class

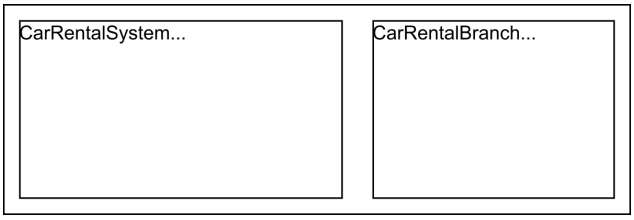


Figure 1.16: The class diagram of the CarRentalSystem and CarRentalBranch classes

- `AccountStatus:` The account status tells about the user account status, i.e., active, closed, canceled, banned, or blocked.
- `ReservationStatus:` The reservation status tells about the reservation state of any vehicle, whether it is in an active state, pending state, confirmed state, completed state, or canceled state.
- `PaymentStatus:` The payment status checks if the customer's payment falls in any of the following stages: unpaid, pending, completed, canceled, or refunded.
- `VanType:` The van type specifies that it can only be of two types: passenger or cargo.
- `CarType:` The car type describes the different types of cars, whether they are economy, compact, intermediate, standard, full-size, premium, or luxury.
- `MotorcycleType:` Similar to the car type, the motorcycle type describes the different types of motorcycles, whether standard, cruiser, touring, sports, off-road, or dual-purpose.
- `TruckType:` The truck type specifies whether the truck is light-duty, medium-duty, or heavy-duty.
- `VehicleLogType:` The vehicle log type describes the type of a particular vehicle log, whether it is an accident, fueling, cleaning service, oil change, repair, or other.

These enumerations can be represented using the following class diagram:

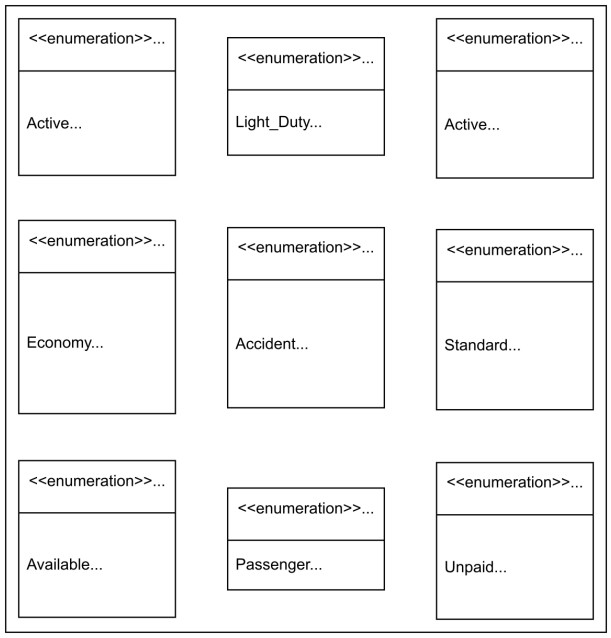


Figure 1.17: Enums in the car rental system

R3: Vehicles can have multiple subtypes. The car type can be economy, luxury, standard, or compact. The van type can be passenger or cargo type. Moreover, the motorcycle type can be cruiser, touring, or sports. The truck type can be light, medium, or high-duty.

4.2 Relationship between the classes

Now, let's discuss the relationships between the main classes in our Car Rental System. Understanding these relationships helps us design for maintainability, extensibility, and clarity.

Association

Association represents a "uses" or "knows about" relationship between two classes. The class diagram will have the following association relationships:

- **One-way association:**
  - The `Account` class should have a one-way association with the `VehicleReservation` class.
  - The `VehicleReservation` class will have a one-way association with the `Vehicle` class. This means they reference `Vehicle` (e.g., by storing a vehicle ID or a reference), but `Vehicle` does not reference them back.
  - The `Fine` class has a one-way association with `Payment`, as a fine is calculated based on or linked to a specific payment, but payment does not necessarily reference a fine.

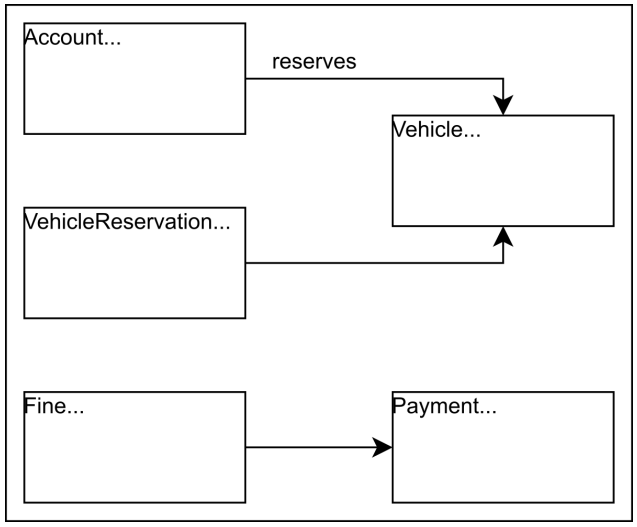


Figure 1.18: The one-way association relationship between classes

- **Two-way association:**

- The `VehicleReservation` class has a two-way association with both `Payment` and `Notification`. A reservation is linked to payment(s) and notifications (for confirmation, reminders, etc.), and payments/notifications can also reference the associated reservation if needed.

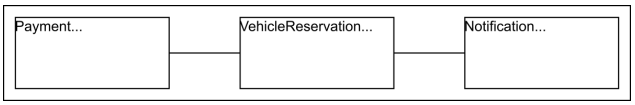


Figure 1.19: The two-way association relationship between classes

Composition

Composition represents a strong "has-a" relationship, where the lifetime of the part is controlled by the whole. The class diagram will have the following composition relationships:

- The `CarRentalBranch` class is composed of multiple `ParkingStall` instances. If the branch is deleted, its parking stalls cease to exist.
- The `Vehicle` class is composed of `VehicleLog` entries. Logs exist only as long as the vehicle exists.

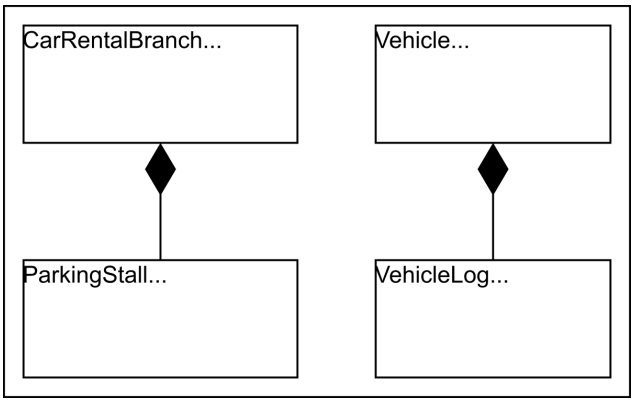


Figure 1.20: The composition relationship between classes

Aggregation

Aggregation is a weaker "has-a" relationship, where the part can exist independently of the whole. The following classes show an aggregation relationship:

- The `CarRentalSystem` class aggregates multiple `CarRentalBranch` instances, meaning branches can exist independently and could potentially be shared or moved between systems.

- The `ParkingStall` and `VehicleCatalog` classes aggregate `Vehicle` instances; a vehicle can be reassigned or exist elsewhere.
- The `VehicleReservation` class aggregates multiple `Equipment` and `Service` instances; equipment and services can be reused across reservations or elsewhere in the system.

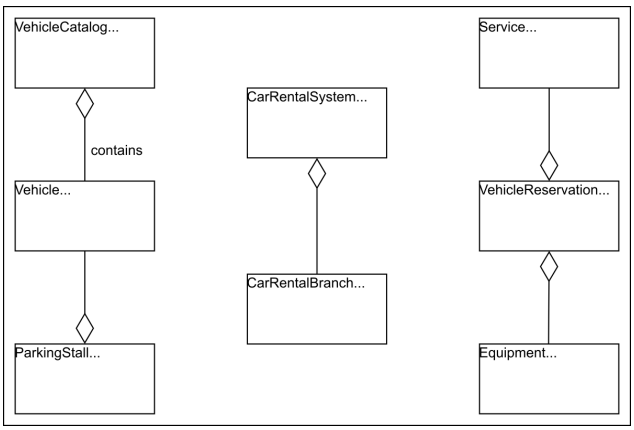


Figure 1.21: The aggregation relationship between classes

Inheritance

Inheritance represents an "is-a" relationship. The following classes show an inheritance relationship:

- The `Receptionist` and `Customer` classes extend the abstract `Account` class, inheriting its fields and methods.
- The `Account` class extends the `Person` class, inheriting personal information.
- The `Car`, `Truck`, `Van`, and `Motorcycle` classes extend the `Vehicle` class, each representing a specialized vehicle type.
- The `DriverService`, `RoadsideAssistance`, and `WiFi` classes extend the abstract `Service` class.
- The `Navigation`, `ChildSeat`, and `SkiRack` classes extend the abstract `Equipment` class.
- The `SmsNotification` and `EmailNotification` classes extend the abstract `Notification` class.
- The `Cash` and `CreditCard` classes extend the abstract `Payment` class.
- The `VehicleCatalog` class implements the `Search` interface, enabling searching for vehicles by type or model.

Note: We've already discussed the inheritance relationship between classes in the component section above one by one.

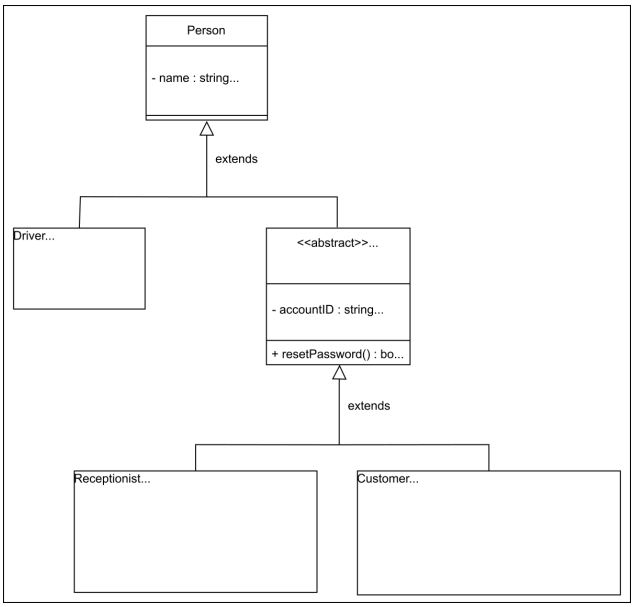


Figure 1.22: The class diagram of the different users of our system

4.3 Class diagram of the car rental system

This section outlines the multiplicity (cardinality) relationships between the main classes in our car rental system. For each relationship, we explain the allowed number of instances on each side and the real-world or design rationale behind the connection. Understanding these relationships is key to modeling how different entities interact and collaborate to support key workflows in the system.

Source	Target	Multiplicity	Reason
`CarRentalSystem`	`CarRentalBranch`	1--0..*	A car rental system manages zero or more branches (locations).
`CarRentalBranch`	`ParkingStall`	1--0..*	Each branch consists of multiple parking stalls where vehicles are parked.
`ParkingStall`	`Vehicle`	0..*--1	Each parking stall can hold one vehicle, but a vehicle could move between stalls; the diagram shows composition at the branch-level, so each stall is unique.

`Vehicle`	`VehicleLog`	1--0..*	Each vehicle has zero or more logs to track events (accidents, maintenance, etc.).
`VehicleCatalog`	`Vehicle`	1--0..*	The catalog contains references to all available vehicles for searching.
`VehicleReservation`	`Vehicle`	1--1	Each reservation is associated with exactly one vehicle.
`VehicleReservation`	`Account`	0..*--1	One customer makes each reservation, but a customer may have multiple reservations (not shown by the arrow but reflected in the code).
`VehicleReservation`	`Equipment`	1--0..*	Each reservation can include zero or more equipment (child seat, ski rack, etc.).
`VehicleReservation`	`Service`	1--0..*	Each reservation can include zero or more optional services (WiFi, driver, roadside assistance).
`VehicleReservation`	`Payment`	1--1	Each reservation involves exactly one payment.
`VehicleReservation`	`Notification`	1--0..*	Each reservation can result in zero or more notifications (confirmation, cancellation, etc.).
`Fine`	`Payment`	0..1--1	A fine is linked to one payment (if incurred), but not all payments result from fines.
`Customer`	`Account`	1--1	Each customer is an account.
`Receptionist`	`Account`	1--1	Each receptionist is an account.

`Account`	`Person`	1--1	Each account corresponds to a person with contact information.

Here's the class diagram of the car rental system:

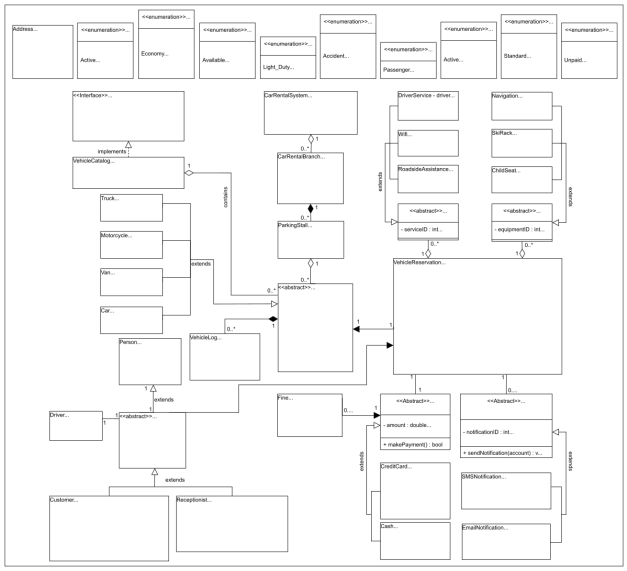


Figure 1.23: The class diagram of the car rental system

4.4 Design pattern

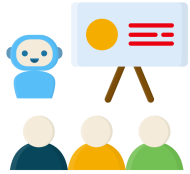
To promote extensibility and adhere to design principles like SRP (Single Responsibility principle) and OCP (Open/Closed principle), we can use the Decorator pattern for dynamic fee calculation and feature extension:

- **DiscountDecorator:** Applies discounts to all types of vehicles.
- **PeakSeasonDecorator:** Adjusts pricing during peak demand.
- **DamageFineDecorator:** Calculates fines for returned vehicles with damage.
- **FuelFineDecorator:** Calculates fines for vehicles returned with a partially filled fuel tank.

These decorators enable you to add or modify price-related behaviors without changing the underlying vehicle, reservation, or payment classes. Other decorators can be introduced as system needs evolve.
Similarly, we can make several other decorators according to the system's needs. These decorators fulfill the SRP and OCP design principles.

4.5 AI-powered trainer

At this stage, everything should be clear. If you encounter any confusion or ambiguity, feel free to utilize the interactive AI-enabled widget below to seek clarification. This tool is designed to help you strengthen your understanding of the concepts.



4.6 Additional requirements

The interviewer can introduce some additional requirements in the car rental system, or they can ask some follow-up questions. Let 's see an example of additional requirements:
Barcode scanner: Each vehicle should have a unique barcode associated with it, and the system should be able to scan the barcode of every vehicle. To fulfill this requirement, we have the class diagram as shown below:

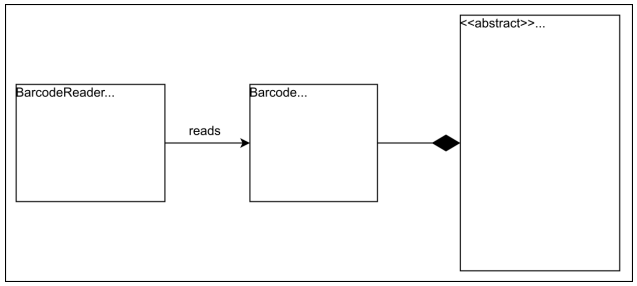


Figure 1.24: The class diagram of barcode scanner functionality

We have completed the class diagram of the car rental system according to the requirements. In the next lesson, let's design the system's sequence diagram.

5 Sequence Diagram for the Car Rental System

Sequence diagrams illustrate the flow of messages and method calls between various actors and objects in the system as they participate in a business process. They are invaluable for visualizing how objects collaborate to complete key scenarios.
In this lesson, we will create sequence diagrams for two essential interactions in the car rental system:

- Vehicle reservation (the customer reserves a vehicle)
- Reservation cancellation (the customer cancels a reservation)

5.1 Vehicle reservation

The sequence diagram for vehicle reservation should have the following actors and objects that will interact with each other:
Actors and objects involved:

- **Actor:** `Customer` (or `Receptionist`, acting on behalf of a customer)
- **Objects:** `VehicleCatalog`, `CarRentalSystem`, `VehicleReservation`, `Payment`, and `Notification`

Interaction steps

1. **Search for vehicle:**
   (a) `Customer` requests to search available vehicles.
   (b) `CarRentalSystem` interacts with the `VehicleCatalog` to retrieve available vehicles.
   (c) `VehicleCatalog` returns a list of available vehicles.
2. **Select and reserve vehicle:**
   (a) `Customer` selects a vehicle to reserve.
   (b) `CarRentalSystem` checks if the vehicle is available.
   (c) If available, the `CarRentalSystem` creates a new `VehicleReservation` object.
3. **Payment processing:**
   (a) `CarRentalSystem` calculates the reservation fee.
   (b) `CarRentalSystem` prompts the `Customer` for payment.
   (c) `Customer` initiates the payment (via `Payment` object).
   (d) `Payment` is processed.
      i. **If successful:**
         A. `CarRentalSystem` updates the reservation status to "confirmed."
         B. `CarRentalSystem` triggers a `Notification` to the customer (reservation confirmation).
         C. `Customer` receives confirmation that the reservation is successful.
      ii. **If unsuccessful:**
         A. `CarRentalSystem` deletes/cancels the reservation.
         B. `CarRentalSystem` triggers a `Notification` to the customer (reservation failed).

C. `Customer` is notified that the reservation is unsuccessful.

4. **Vehicle unavailability:**

   (a) If no vehicles are available, the `CarRentalSystem` notifies the `Customer` accordingly.

Based on the order above, the sequence diagram of a vehicle reservation in a car rental system is provided below:

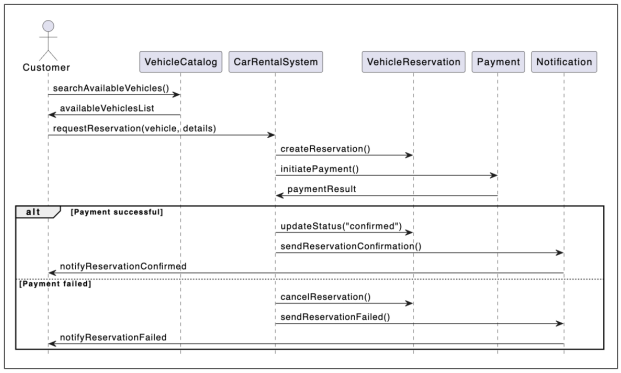


Figure 1.25: The sequence diagram for vehicle reservation

5.2 Sequence challenge: Cancel reservation

You'll help us complete a sequence diagram for a reservation canceled by the member. A skeleton of the cancel reservation sequence diagram is given below:

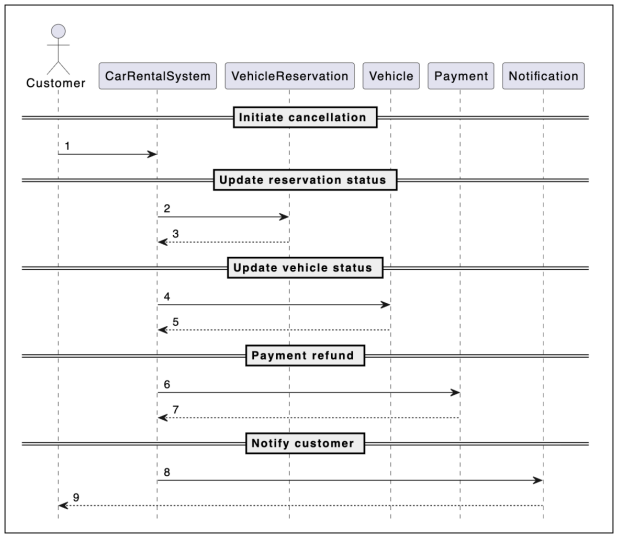


Figure 1.26: The sequence diagram for canceling a reservation

Notice that the arrows in the diagram above are numbered from 1 to 9. Below are the messages between the actor(s) and object(s). Can you rearrange the messages below in the correct sequence of order they should appear in the skeleton of the sequence diagram given above?

Note: If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to view the complete sequence diagram of the cancel reservation interaction.
<img src="/udata/k8qEJqqnpYa/Car Rental.png", width=700>
Next, let's examine the activity diagrams for the car rental system to understand its control flow.

6 Activity Diagram for the Car Rental System

Activity diagrams are a great way to visualize the flow of messages from one activity to another in the system. There can be different activity diagrams that we can create for our car rental system. In this lesson, we will create activity diagrams for the following two activities:

- Vehicle pickup
- **Activity challenge:** Vehicle return

6.1 Vehicle pickup

The states and actions involved in this activity diagram are given below.

States

Initial state: A member with a vehicle booking comes to the "rent a car" reception to pick up the vehicle.
Final state: The guest successfully gets the vehicle, or the system shows a reservation error.

Actions

When a member with a vehicle reservation arrives at the rent-a-car reception, the receptionist validates the reservation and updates the vehicle status.
Based on the order above, the activity diagram of the vehicle pickup from the car rental facility is given below.

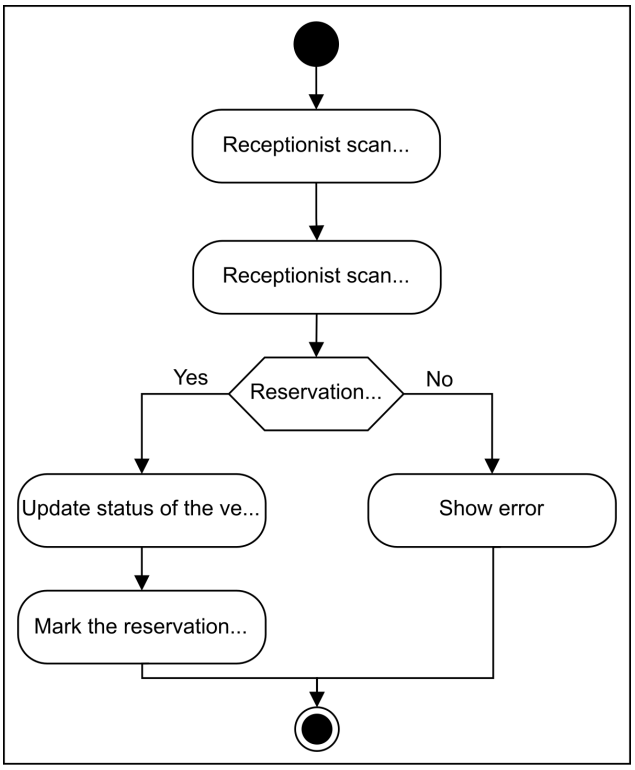


Figure 1.27: The activity diagram for vehicle pickup

6.2 Activity challenge: Vehicle return

You will help us create an activity diagram of a member who wants to return the vehicle. Here is the activity diagram structure:

Notice that the actions in the diagram above are numbered from 1 to 9. The slots below represent the activities, and the arrows represent the flow from one activity to another. Can you rearrange the slots below in the correct order they should appear in the activity diagram above?

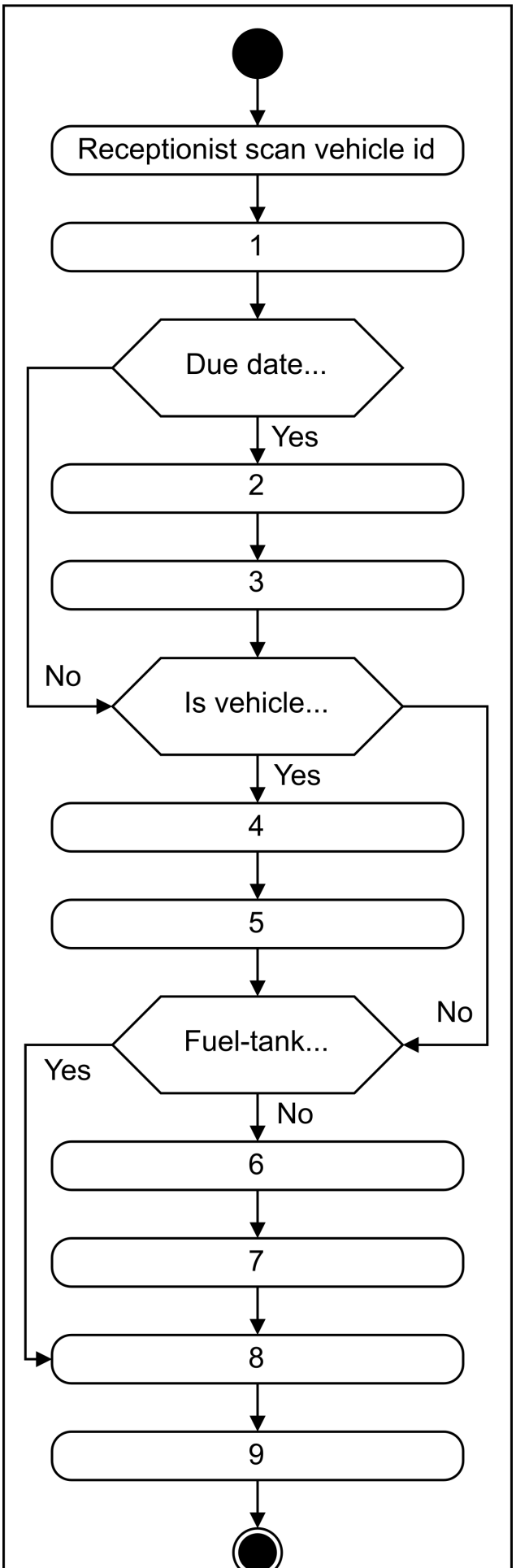
> **Note:** If you're stuck, click the "Show Solution" button.

Alternatively, you can also click the "Show complete diagram" button below to view the complete sequence diagram of the cancel reservation interaction.

We've looked at some of our car rental system's activity diagrams. In the next lesson, we will present the code for our designed classes in some of the most popular languages.

7 Code for the Car Rental System

We've reviewed different aspects of the car rental system and observed the attributes attached to the problem using various UML diagrams. Let 's explore the more practical side of things, where we will work on implementing the car rental system using multiple languages. This is


Receptionist scan vehicle id
1
Due date...
Yes
2
3
No
Is vehicle...
Yes
4
5
No
Fuel-tank...
Yes
No
6
7
8
9

usually the last step in an object-oriented design interview process.
We have chosen the following languages to write the skeleton code of the different classes present in the car rental system:

- Java
- C#
- Python
- C++
- JavaScript

If you want to skip directly to the implementation and see the complete workflow, simply scroll down or click Executable Code: Car Rental System.

7.1 Car rental system classes

This section will provide the skeleton code of the classes designed in the class diagram lesson.

> **Note:** For simplicity, we are not defining getter and setter functions. The reader can assume that all class attributes are private, accessed through their respective public getter methods, and modified only through their public methods.

Enumerations

First, we will define all the enumerations(A special data type which contains a set of predefined constants.) required in the car rental system. According to the class diagram, seven enumerations are used in the system, i.e., `VehicleStatus`, `AccountStatus`, `ReservationStatus`, `PaymentStatus`, `VanType`, `CarType`, and `VehicleLogType`. The code to implement these enumerations is as follows:

> **Note:** JavaScript does not support enumerations, so we will use the Object.freeze() method as an alternative that freezes an object and prevents further modifications.

Enum definitions

```
public enum MotorcycleType {
    STANDARD, CRUISER, TOURING, SPORTS, OFF_ROAD, DUAL_PURPOSE
}

public enum AccountStatus {
    ACTIVE, CLOSED, CANCELED, BLACKLISTED, BLOCKED
}

public enum ReservationStatus {
    ACTIVE, PENDING, CONFIRMED, COMPLETED, CANCELED
}
```

```
public enum PaymentStatus {
    UNPAID, PENDING, COMPLETED, CANCELED, REFUNDED
}

public enum VanType {
    PASSENGER, CARGO
}

public enum CarType {
    ECONOMY, COMPACT, INTERMEDIATE, STANDARD, FULL_SIZE, PREMIUM, LUXURY
}

public enum MotorcycleType {
    STANDARD, CRUISER, TOURING, SPORTS, OFF_ROAD, DUAL_PURPOSE
}

public enum TruckType {
    LIGHT_DUTY, MEDIUM_DUTY, HEAVY_DUTY
}

public enum VehicleLogType {
    ACCIDENT, FUELING, CLEANING_SERVICE, OIL_CHANGE, REPAIR, OTHER
}
```

Address, person, and driver

This section contains the `Address`, `Person`, and `Driver` classes, where the first two classes are used as a custom data type. The implementation of these classes is shown below:

The Address, Person, and Driver classes

```
public class Address {
    private String streetAddress;
    private String city;
    private String state;
    private int zipCode;
    private String country;

    public Address(String street, String city, String state, int zip, String
        country) {
        this.streetAddress = street;
        this.city = city;
        this.state = state;
        this.zipCode = zip;
        this.country = country;
    }
    // Getters...
}

public abstract class Person {
    private String name;
```

```
    private Address address;
    private String email;
    private String phoneNumber;

    // Setters & getters...
}

public class Driver extends Person {
    private int driverId;
    // Setters/getters...
}
```

Account

`Account` is an abstract class that represents the various people or actors that can interact with the system. There are two types of accounts: receptionist and customer. The implementation of `Account` and its subclasses is shown below:

Account and its derived classes

```
public abstract class Account extends Person {
    private String accountId;
    private String password;
    private AccountStatus status;

    public Account() {}
    // Optional: full constructor
    // Setters/getters...
    public abstract boolean resetPassword();
}

public class Customer extends Account {
    private String licenseNumber;
    private Date licenseExpiry;

    // Setters/getters...
    @Override
    public boolean resetPassword() {
        this.setPassword(java.util.UUID.randomUUID().toString());
        return true;
    }
}

public class Receptionist extends Account {
    private Date dateJoined;
    private List<Customer> customerList = new ArrayList<>();
    // Add/search/remove customer methods...
    @Override
    public boolean resetPassword() {
        this.setPassword(java.util.UUID.randomUUID().toString());
        return true;
    }
```

```
}
```

Vehicle

`Vehicle` will be another abstract class, which serves as a parent for four different types of vehicles: `Car`, `Van`, `Truck`, and `MotorCycle`. The definition of the `Vehicle` and its child classes is given below:

Vehicle and its child classes

```
import java.util.*;
public abstract class Vehicle {
    private String vehicleId;
    private String licensePlateNumber;
    private int passengerCapacity;
    private VehicleStatus status;
    private String model;
    private int manufacturingYear;
    private List<VehicleLog> log = new ArrayList<>();
    // Setters/getters, reserveVehicle, returnVehicle
}

public class Car extends Vehicle { private CarType carType; /* Setters/getters...
    */ }
public class Van extends Vehicle { private VanType vanType; /* Setters/getters...
    */ }
public class Truck extends Vehicle { private TruckType truckType; /* Setters/
    getters... */ }
public class Motorcycle extends Vehicle { private MotorcycleType motorcycleType; /*
     Setters/getters... */ }
```

Equipment

`Equipment` is an abstract class, and this section represents different equipment: `Navigation`, `ChildSeat`, and `SkiRack` added in the reservation. The code to implement these classes is shown below:

Equipment and its derived classes

```
public abstract class Equipment {
    private int equipmentId;
    private int price;
    // Setters/getters...
}

public class Navigation extends Equipment { /* Extend if needed */ }
public class ChildSeat extends Equipment { /* Extend if needed */ }
public class SkiRack extends Equipment { /* Extend if needed */ }
```

Service

`Service` is an abstract class, and this section represents different services: `DriverService`, `RoadsideAssistance`, and `Wi-Fi` added to the reservation. The code to implement these classes is shown below:

Service and its derived classes

```
public abstract class Service {
    private int serviceId;
    private int price;
    // Setters/getters...
}

public class DriverService extends Service {
    private int driverId;
    // Setters/getters...
}
public class RoadsideAssistance extends Service { /* Extend if needed */ }
public class WiFi extends Service { /* Extend if needed */ }
```

Payment

The `Payment` class is another abstract class, with the `Cash` and `CreditCard` classes as its children. This takes in the `PaymentStatus` enum to keep track of the payment status. The definition of this class is provided below:

Payment and its child classes

```
import java.util.*;
public abstract class Payment {
    private double amount;
    private Date timestamp;
    private PaymentStatus status;
    // Setters/getters...
    public abstract boolean makePayment();
}

public class Cash extends Payment {
    @Override
    public boolean makePayment() { setStatus(PaymentStatus.COMPLETED); return true;
        }
}

public class CreditCard extends Payment {
    private String nameOnCard;
    private String cardNumber;
    private String billingAddress;
    private int code;
    // Setters...
    @Override
```

```
    public boolean makePayment() { setStatus(PaymentStatus.COMPLETED); return true;
        }
}
```

Vehicle log and vehicle reservation

`VehicleLog` is a class responsible for keeping track of all the events related to a vehicle. `VehicleReservation` is a class responsible for managing the reservation of vehicles. The implementation of this class is given below:

The VehicleLog and VehicleReservation classes

```
import java.util.*;
public class VehicleLog {
    private int logId;
    private VehicleLogType logType;
    private String description;
    private Date creationDate;
    // Constructor, getters...
}

public class VehicleReservation {
    private int reservationId;
    private String customerId;
    private String vehicleId;
    private Date creationDate;
    private ReservationStatus status;
    private Date dueDate;
    private Date returnDate;
    private String pickupLocation;
    private String returnLocation;
    private List<Equipment> equipments = new ArrayList<>();
    private List<Service> services = new ArrayList<>();
    // Setters/getters, addEquipment(), addService(), etc.
}
```

Notification

The `Notification` class is another abstract class responsible for sending notifications, with the `SMSNotification` and `EmailNotification` classes as its children. The implementation of this class is shown below:

Notification and its derived classes

```
import java.util.*;
public abstract class Notification {
    private int notificationId;
    private Date createdOn;
```

```
    private String content;
    public void setContent(String c) { content = c; }
    public String getContent() { return content; }
    public abstract void sendNotification(Account account);
}

public class SmsNotification extends Notification {
    @Override
    public void sendNotification(Account account) {
        System.out.println("SMS to " + account.getName() + ": " + getContent());
    }
}

public class EmailNotification extends Notification {
    @Override
    public void sendNotification(Account account) {
        System.out.println("Email to " + account.getName() + ": " + getContent());
    }
}
```

Parking stall and fine

`ParkingStall` is a class used to locate vehicles in the car rental branch, while the `Fine` class represents the fine applied for payment. The implementation of these classes is given below:

The Parking stall and fine classes

```
public class ParkingStall {
    private int stallId;
    private String locationIdentifier;
    // Constructor, getters...
}


public class Fine {
    private double amount;
    private String reason;
    public void setAmount(double a) { amount = a; }
    public double getAmount() { return amount; }
    public void setReason(String r) { reason = r; }
    public String getReason() { return reason; }
    public double calculateFine() { return amount; }
}
```

Search interface and vehicle catalog

`Search` is an interface, and the `VehicleCatalog` class is used to implement the search interface to help in vehicle searching. The code to perform this function is presented below:

The Search interface and the VehicleCatalog class

```java
import java.util.*;
public interface Search {
    List<Vehicle> searchByType(String type);
    List<Vehicle> searchByModel(String model);
}

public class VehicleCatalog implements Search {
    private HashMap<String, List<Vehicle>> vehicleTypes = new HashMap<>();
    private HashMap<String, List<Vehicle>> vehicleModels = new HashMap<>();
    // addVehicle(), searchByType(), searchByModel()
}
```

Car rental system and car rental branch

The `CarRentalSystem` class is the base class of the system that is used to represent the whole car rental system (or the top-level classes of the system). `CarRentalBranch` represents the single branch of the system. The implementation of these classes is given below:

The CarRentalSystem and CarRentalBranch classes

```java
import java.util.*;
public class CarRentalBranch {
    private String name;
    private Address address;
    private List<ParkingStall> stalls;
    // Constructor, getters...
}

public class CarRentalSystem {
    // Singleton pattern
    private String name;
    private List<CarRentalBranch> branches = new ArrayList<>();
    private static CarRentalSystem system = null;
    private CarRentalSystem() {}
    public static CarRentalSystem getInstance() {
        if (system == null) system = new CarRentalSystem();
        return system;
    }
    // addNewBranch(), getBranches()...
}
```

7.2 Executable code: Car rental system

Below is a fully self-contained, runnable program in Java, C#, C++, Python, and JavaScript that demonstrates the core workflows of the Car Rental system. The system's main driver code resides in `Driver.java`, `Driver.cs`, `Driver.py`, `Driver.cpp`, and `Driver.js` for all respective languages. You can click the "Run" button to execute the codes.

What this code shows

- **System Initialization:** Sets up branches, adds vehicles to the inventory, and registers a customer.
- **Scenario 1 (Vehicle search):** The customer searches the inventory for available vehicles.
- **Scenario 2 (Reservation and add-ons):** The customer makes a reservation and adds equipment (Child Seat) and an extra service (Driver).
- **Scenario 3 (Payment):** The customer completes payment for the reservation.
- **Scenario 4 (Notification):** The customer receives an email notification for reservation confirmation.
- **Scenario 5 (Pickup and return):** The customer picks up and returns the vehicle.
- **Scenario 6 (Fine calculation):** The system checks for overdue returns and applies a fine if needed.
- **Scenario 7 (Reservation history):** Displays all reservations made by the customer.

Hint: Try customizing the code!
This code is designed for experimentation, not just reading. You can edit, extend, or modify any part of the example.

Executable Code: Car Rental System

```
import java.util.*;

public class CarRentalSystemDriver {
    public static void main(String[] args) {
        System.out.println("==============================================");
        System.out.println(" CAR RENTAL SYSTEM DEMO");
        System.out.println("==============================================
");

        // ------------------------------------------------
        // 1. Branch Setup
        // ------------------------------------------------
        System.out.println("1. Branch Setup");
        CarRentalSystem rentalSystem = CarRentalSystem.getInstance();
        CarRentalBranch branch1 = new CarRentalBranch(
                "Airport Branch",
                new Address("123 Main St", "Seattle", "WA", 98101, "USA"),
                new ArrayList<>());
        rentalSystem.addNewBranch(branch1);
        System.out.println(" -> Branch added: " + branch1.getName() + " (" + branch1
            .getLocation() + ")
");

        // ------------------------------------------------
```

```
        // 2. Add Vehicles
        // ------------------------------------------------
        System.out.println("2. Adding Vehicles to Inventory");
        Car car1 = new Car();
        car1.setVehicleId("C1001");
        car1.setModel("Toyota Corolla");
        car1.setManufacturingYear(2022);
        car1.setCarType(CarType.ECONOMY);
        car1.setStatus(VehicleStatus.AVAILABLE);

        Van van1 = new Van();
        van1.setVehicleId("V1001");
        van1.setModel("Ford Transit");
        van1.setManufacturingYear(2021);
        van1.setVanType(VanType.PASSENGER);
        van1.setStatus(VehicleStatus.AVAILABLE);

        VehicleCatalog catalog = new VehicleCatalog();
        catalog.addVehicle(car1);
        catalog.addVehicle(van1);

        System.out.println(" -> Vehicles added: ");
        System.out.println(" - " + car1.getModel() + " (ID: " + car1.getVehicleId()
            + ")");
        System.out.println(" - " + van1.getModel() + " (ID: " + van1.getVehicleId()
            + ")
");

        // ------------------------------------------------
        // 3. Customer Registration
        // ------------------------------------------------
        System.out.println("3. Customer Registration & Login");
        Customer customer1 = new Customer();
        customer1.setAccountId("U123");
        customer1.setName("Alice Smith");
        customer1.setEmail("alice@email.com");
        customer1.setLicenseNumber("D1234567");

        Calendar expiryCal = Calendar.getInstance();
        expiryCal.set(2027, Calendar.FEBRUARY, 1);
        customer1.setLicenseExpiry(expiryCal.getTime());
        customer1.setStatus(AccountStatus.ACTIVE);

        System.out.println(" -> [LOGIN] Customer: " + customer1.getName() + " (ID: "
             + customer1.getAccountId() + ")");
        System.out.println(" -> Driver License #: " + customer1.getLicenseNumber() +
               " (Expires: " + customer1.getLicenseExpiry() + ")
");

        // ------------------------------------------------
        // 4. Vehicle Search by Customer
        // ------------------------------------------------
        System.out.println("4. Vehicle Search");
        System.out.println(" == Vehicle Inventory Search Results ==");
        List<Vehicle> cars = catalog.searchByType("CAR");
        if (!cars.isEmpty()) {
```

```
            System.out.println(" -> Found " + cars.size() + " car(s) in inventory:")
                ;
            for (Vehicle v : cars) {
                System.out.println(" -> Model: " + v.getModel() +
                        " | ID: " + v.getVehicleId() +
                        " | Year: " + v.getManufacturingYear() +
                        " | Status: " + v.getStatus());
            }
        } else {
            System.out.println(" -> No cars found in inventory.");
        }
        System.out.println();

        // ------------------------------------------------
        // 5. Make a Reservation
        // ------------------------------------------------
        System.out.println("5. Reservation");
        VehicleReservation reservation = new VehicleReservation();
        reservation.setReservationId(1);
        reservation.setCustomerId(customer1.getAccountId());
        reservation.setVehicleId(car1.getVehicleId());
        reservation.setCreationDate(new Date());
        reservation.setStatus(ReservationStatus.PENDING);
        reservation.setPickupLocation("Airport Branch");
        reservation.setReturnLocation("Airport Branch");

        Calendar dueCal = Calendar.getInstance();
        dueCal.add(Calendar.DAY_OF_MONTH, 3);
        reservation.setDueDate(dueCal.getTime());

        List<VehicleReservation> allReservations = new ArrayList<>();
        allReservations.add(reservation);

        car1.reserveVehicle();
        System.out.println(" -> Reservation created for " + customer1.getName() +
                " | Vehicle: " + car1.getModel() +
                " | Pickup: " + reservation.getPickupLocation() +
                " | Return: " + reservation.getReturnLocation() +
                " | Due: " + reservation.getDueDate());

        // ------------------------------------------------
        // 6. Add Equipment and Services
        // ------------------------------------------------
        System.out.println("
6. Add-ons: Equipment & Services");
        ChildSeat seat = new ChildSeat();
        seat.setEquipmentId(10);
        seat.setPrice(15);
        reservation.addEquipment(seat);

        DriverService driverService = new DriverService();
        driverService.setServiceId(20);
        driverService.setPrice(50);
        driverService.setDriverId(222);
        reservation.addService(driverService);
```

```
        System.out.println(" -> Equipment added: Child Seat (ID: " + seat.
            getEquipmentId() + ", Price: $" + seat.getPrice() + ")");
        System.out.println(" -> Service added: Driver (Driver ID: " + driverService.
            getDriverId() + ", Price: $" + driverService.getPrice() + ")
");

        // ------------------------------------------------
        // 7. Payment Processing
        // ------------------------------------------------
        System.out.println("7. Payment Processing");
        reservation.setStatus(ReservationStatus.CONFIRMED);
        Payment payment = new CreditCard();
        payment.setAmount(200);
        payment.setTimestamp(new Date());
        payment.setStatus(PaymentStatus.PENDING);
        System.out.println(" -> Processing payment of $" + payment.getAmount() + "
            ...");
        boolean paymentSuccess = payment.makePayment();

        if (paymentSuccess) {
            System.out.println(" -> Payment completed successfully for reservation #
                " + reservation.getReservationId());
        } else {
            System.out.println(" -> Payment failed!");
        }
        System.out.println();

        // ------------------------------------------------
        // 8. Notification (Email)
        // ------------------------------------------------
        System.out.println("8. Notification");
        Notification notify = new EmailNotification();
        notify.setContent("Your reservation is confirmed!");
        notify.sendNotification(customer1);
        System.out.println();

        // ------------------------------------------------
        // 9. Vehicle Pickup
        // ------------------------------------------------
        System.out.println("9. Vehicle Pickup");
        System.out.println(" -> " + customer1.getName() + " picked up the vehicle: "
             +
                car1.getModel() + " (" + car1.getVehicleId() + ")
");

        // ------------------------------------------------
        // 10. Vehicle Return
        // ------------------------------------------------
        System.out.println("10. Vehicle Return");
        Calendar returnCal = Calendar.getInstance();
        returnCal.add(Calendar.DAY_OF_MONTH, 3); // Returned on due date
        car1.returnVehicle();
        reservation.setReturnDate(returnCal.getTime());
        reservation.setStatus(ReservationStatus.COMPLETED);

        System.out.println(" -> Vehicle returned on: " + reservation.getReturnDate()
```

```
            );
        System.out.println(" -> Reservation status is now: " + reservation.getStatus
            () + "
");

        // ------------------------------------------------
        // 11. Overdue Fine Check
        // ------------------------------------------------
        System.out.println("11. Fine Calculation");
        Date expectedReturn = reservation.getDueDate();
        Date actualReturn = reservation.getReturnDate();
        if (actualReturn.after(expectedReturn)) {
            Fine fine = new Fine();
            fine.setAmount(100);
            fine.setReason("Late return");
            System.out.println(" -> [FINE] Fine imposed for late return: $" + fine.
                getAmount());
            Notification fineNotify = new SmsNotification();
            fineNotify.setContent("You have been fined for late return.");
            fineNotify.sendNotification(customer1);
        } else {
            System.out.println(" -> [FINE] No fine. Vehicle was returned on time.");
        }
        System.out.println();

        // ------------------------------------------------
        // 12. Reservation History for Customer
        // ------------------------------------------------
        System.out.println("12. Reservation History for Customer: " + customer1.
            getName());
        System.out.println(" -------------------------------------");
        for (VehicleReservation r : allReservations) {
            if (r.getCustomerId().equals(customer1.getAccountId())) {
                System.out.println(" -> Reservation ID: " + r.getReservationId() +
                        " | Vehicle: " + r.getVehicleId() +
                        " | Status: " + r.getStatus() +
                        " | Pickup: " + r.getPickupLocation() +
                        " | Returned: " + r.getReturnDate());
            }
        }
        System.out.println("============================================");
        System.out.println(" END OF DEMO");
        System.out.println("============================================");
    }
}
```

7.3 Wrapping up

In this chapter, we've explored the complete design of a car rental system. We 've also seen how a basic system can be visualized using various UML diagrams and designed using object-oriented principles and design patterns.

8 Summary: Designing a Car Rental System

Now that you've completed the car rental system case study, let's take a moment to reflect on and consolidate what we've learned. We 'll revisit the key system requirements, identify the core classes along with their responsibilities and relationships, and highlight the major design principles applied. We 'll also examine how objects interact within the system and walk through the overall workflow to understand how the components come together to achieve the desired functionality.

8.1 Key requirements

The following are the primary functional and operational requirements for the car rental system:

1. Support two user roles: customer and receptionist.
2. Manage multiple vehicle types: cars, trucks, vans, motorcycles (with subtypes).
3. Track reservations, customer info, rental history, and vehicle logs.
4. Support the reservation life cycle: make, update, cancel, pick up, and return.
5. Allow optional services and equipment (e.g., Wi-Fi, child seat).
6. Handle late returns and issue fines automatically.
7. Enable vehicle search by type, model, features, or location.
8. Maintain multi-branch support with branch-specific inventories.
9. Track parking stall availability at each branch.
10. Support secure payments and refunds via multiple methods.
11. Allow payment at reservation time or vehicle return.

8.2 System actors

This system involves several actors with distinct roles and capabilities:

- **Customer:** Registers, searches for vehicles, makes and updates reservations, selects services/equipment, makes payments, and handles vehicle pickup/return.
- **Receptionist:** Performs customer operations on their behalf and manages vehicle inventory, logs, and payment processing at the branch.

8.3 Important classes and relationships

The system design includes several key classes that collaborate to fulfill car rental operations.

Class	Description	Important Relationships
`Address`	Represents a physical address used by persons and branches	Used by `Customer`, `Receptionist`, and `CarRentalBranch`
`Person`	Abstract base class for all system users	Extended by `Account`
`Account`	Abstract class managing user login and access	Extended by `Customer` and `Receptionist`. Associated with `VehicleReservation`
`Customer`	Concrete class representing end users	Inherits from `Account`; handles add/cancel and get reservation
`Receptionist`	Admin-level account managing branches and reservations	Inherits from `Customer`; manages Vehicle, `VehicleLog`, and `Payment`
`DriverService`	Represents an additional driver assigned to a reservation	Associated with `Service`
`Vehicle`	Abstract class for rentable units	Composed of `VehicleLog`; extended by `Car`, `Truck`, `Van`, `Motorcycle`
`Equipment`	Abstract class for optional add-ons	Subclasses include `Navigation`, `ChildSeat`, `SkiRack`
`Service`	Abstract class for value-added services	Subclasses include `DriverService`, `WiFi`, `RoadsideAssistance`
`Notification`	Abstract class to notify customers	Subclasses include `SMSNotification`, `EmailNotification`
`ParkingStall`	Represents vehicle storage units at a branch	Composed in `CarRentalBranc` h; aggregates `Vehicle`
`VehicleLog`	Tracks maintenance, repairs, and events	Composed in `Vehicle`
`VehicleReservation`	Manages reservation state and associated services/equipment	Associated with `Payment`, `Vehicle`, `Notification`
`Payment`	Abstract class for processing fees	Subclasses: `CreditCard`, `Cash`; associated with `Fine`
`Fine`	Applies late fees or penalties	One-way association with `Payment`
`Search`	Interface defining search operations	Implemented by `VehicleCatalog`

`VehicleCatalog`	Enables search by model/type	Aggregates `Vehicle`
`CarRentalBranch`	Represents a system location	Composed of `ParkingStall`
`CarRentalSystem`	Core orchestrator of branches, reservations, inventory, and search	Aggregates `CarRentalBranch`

8.4 Design highlights

In this section, we outline the design approach and the application of SOLID principles and design patterns to ensure modularity, flexibility, and scalability.

Design approach

The system follows a bottom-up approach, beginning with simple entities like `Vehicle`, `Customer`, and `Reservation`, and building up to high-level modules like inventory and payment management. It integrates subsystems to produce a cohesive, modular application.

Design principles

The system applies SOLID design principles as follows:

- **Single Responsibility principle (SRP):** Each class (e.g., `Payment`, `Notification`, `Reservation`) encapsulates one well-defined responsibility.
- **Open/Closed principle (OCP):** Services, equipment, and decorators can be extended without altering core logic.
- **Liskov Substitution principle (LSP):** Subclasses (e.g., `Car`, `Truck`) can be used wherever their base `Vehicle` type is expected.
- **Interface Segregation principle (ISP):** Specific interfaces like `Search` prevent dependency on unused operations.
- **Dependency Inversion principle (DIP):** High-level modules like `CarRentalSystem` depend on abstractions (`Search`, `Payment`, etc.), not implementations.

Design patterns

Design patterns help structure the system's complexity:

- **Decorator pattern:** Used for dynamic fee calculation (e.g., `DiscountDecorator`, `DamageFineDecorator`) without modifying core logic.
- **Interface pattern:** Employed via the `Search` interface to allow flexible querying.

8.5 Object interactions

Here, we describe the key message exchanges between objects that model real-time behaviors within the system.

- **Vehicle reservation:**
  - Customer searches for vehicles via `VehicleCatalog`.
  - Available vehicles are returned from `CarRentalSystem`.
  - Customer selects a vehicle; system checks availability and creates a `VehicleReservation`.
  - The system calculates fees and prompts for payment.
  - Upon success, the reservation is confirmed, and a `Notification` is sent.
- **Reservation cancellation:**
  - Customer requests cancellation.
  - System verifies policy and updates reservation status.
  - Any applicable refund or fine is processed via `Payment`.
  - A cancellation `Notification` is sent.
- **Overdue return:**
  - Upon late return, the system detects overdue via the `Reservation`.
  - A `Fine` is applied.
  - A `Notification` alerts the customer about the overdue charge.
- **Add services/equipment:**
  - During or after the reservation, the customer selects optional items.
  - `Service` or `Equipment` objects are added to the reservation.
  - Total cost is recalculated.

8.6 System workflow

This section outlines the main workflows that occur in the system, as illustrated in the activity diagrams.

- **Vehicle reservation:**
  - Customer searches vehicle inventory by date, location, or model.
  - Selects and reserves a vehicle with optional services/equipment.

  - Makes payment during the reservation or upon return.

- **Vehicle pickup:**
  - The customer arrives at the branch.
  - The receptionist verifies the reservation and updates the vehicle status.
  - The vehicle is handed over to the customer.

- **Vehicle return:**
  - Vehicle is returned at the designated branch.
  - The receptionist inspects for damage or refueling.
  - Applicable fines are assessed.
  - Final payment is collected, and vehicle status is updated.

- **Optional services handling:**
  - During reservation updates, the customer selects add-ons.
  - Services like `Wi-Fi` or Roadside Assistance are added to the reservation object.
  - Payment is adjusted accordingly.

- **Notifications workflow:**
  - System sends email/SMS for confirmations, cancellations, and overdue alerts.

8.7

9 Key Insights and Tips: Designing a Car Rental System

Congratulations on completing the case study for designing a car rental system. This lesson expands on your understanding by covering common design pitfalls, sharing practical interview tips, and presenting a quiz to test your grasp of key concepts. You 'll also find related case studies to help deepen your knowledge of object-oriented design in real-world systems.

9.1 Common mistakes

Review this list to avoid these specific pitfalls when designing a car rental system:

- **Overlooking vehicle log management:** Forgetting detailed logs like maintenance, accident history, or status changes hinders effective fleet tracking. Each `Vehicle` should contain a composed `VehicleLog` list to record life cycle events.
- **Ignoring branch-specific inventory management:** Failing to model vehicle availability per branch disrupts location-based operations. Each `CarRentalBranch` should manage its own `ParkingStall` and vehicle inventory.
- **Missing proper payment and fine management integration:** Not linking fines to overdue returns, fuel levels, or damage weakens accountability. Integrate the `Fine` class with the `VehicleReservation` and `Payment` classes to automate penalties.
- **Neglecting reservation modification and cancellation logic:** The system becomes inconsistent if update or cancel flows aren't defined. Support `updateReservation()` and `cancelReservation()` operations with status updates and notifications.
- **Not managing additional services or equipment effectively:** Skipping links between reservations and optional services causes booking ambiguity. Ensure `VehicleReservation` aggregates `Service` and `Equipment` instances for flexibility.

9.2 Key interview tips

Use these tips to enhance your car rental system design during the interview:

Tip	Explanation
Track vehicle and reservation status with enums.	Define clear state transitions using `VehicleStatus` and `ReservationStatus` enums to prevent undefined or conflicting states.
Implement the Search interface.	Support search by vehicle type or model using a flexible `VehicleCatalog` class.
Design for branch-level granularity.	Each `CarRentalBranch` should manage its own `ParkingStalls`, vehicle inventory, and logs.
Capture overdue logic via notification and fine system.	Automate overdue alerts and fines with `Notification` and `Fine` classes.
Decouple the reservation from the payment.	Link `VehicleReservation` and `Payment` via association for clear, modular handling.

Table 1.1: Tips for Designing and Explaining a Car Rental System

9.3 Test your understanding

Test your understanding of key system design decisions in this car rental system case:

Designing a Car Rental System

Question 1: What is the correct way to model additional vehicle services like Wi-Fi or roadside assistance?

- Use a flag in the `Reservation` class.
- Add methods directly to the `Vehicle` class.
- **[CORRECT]** Create subclasses from an abstract `Service` class.
  Explanation: Services are modeled as subclasses of an abstract `Service` class to allow extensibility.
- Store as strings in a list.

Question 2: Which relationship best models how `CarRentalBranch` and its `ParkingStall` should interact in an object-oriented design?

- Aggregation, because parking stalls can be shared across branches.
- Inheritance, because parking stalls are specialized types of branches.
- Association, because the branch only needs to reference stalls.
- **[CORRECT]** Composition, because stalls should not exist independently of their branch.
  Explanation: Composition is appropriate here because a `ParkingStall` is tightly bound to a `CarRentalBranch`. If the branch is deleted, its stalls should also be removed, indicating strong ownership.

Question 3: Why is a `VehicleCatalog` class used in the design?

- To store customer payment details
- To maintain logs of vehicle maintenance
- **[CORRECT]** To implement search functionality for vehicles
  Explanation: `VehicleCatalog` is responsible for enabling flexible vehicle search by type or model.
- To assign drivers to reservations

Question 4: Why should the `VehicleReservation` class aggregate `Service` and `Equipment` objects?

- Because services and equipment are stored globally and reused.
- Because reservations and services share the same life cycle.
- To prevent direct associations between customers and equipment.
- **[CORRECT]** To allow flexible addition of optional features without tightly coupling them to vehicles.

  Explanation: Aggregation allows `VehicleReservation` to flexibly include optional `Service` and `Equipment` instances without creating rigid dependencies, supporting extensibility.

Question 5: What is the best reason to use enumerations like `ReservationStatus` or `VehicleStatus` in this system?

- They speed up database queries by caching.
- They simplify UI development.
- **[CORRECT]** They make state transitions explicit and reduce ambiguity in system behavior.

  Explanation: Enumerations are key to maintaining well-defined state transitions and ensuring consistency across the system, which is crucial in complex workflows like reservations and vehicle availability.
- They allow for faster vehicle filtering by type.

9.4 Related case studies

Broaden your understanding by exploring these related scenarios:

- **Designing a hotel management system:** Similar to the car rental system in terms of bookings, multi-role access (guests, receptionist, maintenance), and room/service management. It reinforces modeling of reservations, availability checks, and life cycle updates of physical assets.
- **Designing an airline management system:** Reinforces concepts like seat allocation, route management, dynamic pricing, and booking cancellations. Like car rentals, it involves managing inventory distributed across locations with time-bound reservations.
- **Designing a restaurant management system:** Focuses on table reservations, menu-based service selection, and payment workflows, mirroring the optional service/equipment flows and user-specific bookings in car rentals.

- **Designing an online stock brokerage system:** While domain-specific, this case sharpens understanding of user-account isolation, transaction tracking, and notification handling—patterns essential for billing and overdue alerts in a car rental setup.
- **Library management system:** A system that includes user-based lending, late-return fines, inventory tracking, and searchability. It's conceptually close to car rentals in handling time-based asset usage, role differentiation, and penalties.